

Proyecto de PLN – Formalismos

Analizador de Requerimientos de Dispositivos · Universidad del Valle

Integrantes: Pedro Bernal Londoño y Santiago Reyes Rodríguez

1. La gramática del sistema $G = (V, T, P, S)$ (gramatica.py)

El sistema entiende frases en español sobre dispositivos tecnológicos. Para eso definimos una gramática formal en gramatica.py que especifica qué frases son válidas. La gramática tiene cuatro partes:

- V (símbolos no terminales): las categorías que usamos internamente, como INTENCION, REQUERIMIENTO, SUJETO, MODIFICADORES, COMPLEMENTO, MARCA, GAMA, ADJETIVO, entre otros (23 en total).
- T (terminales): las palabras reales que puede escribir el usuario, como busco, quiero, celular, samsung, gama, alta, para, jugar, etc. (alrededor de 70 palabras en el léxico).
- S (símbolo inicial): la oración completa, representada por S .
- P (producciones): las reglas que dicen cómo se arma una oración válida.

Las reglas más importantes son:

- $S \rightarrow \text{INTENCION REQUERIMIENTO} \mid \text{INTENCION REQUERIMIENTO COMPLEMENTO} \mid \text{REQUERIMIENTO}$
- $\text{REQUERIMIENTO} \rightarrow \text{ARTICULO SUJETO MODIFICADORES} \mid \text{ARTICULO ADJETIVO SUJETO MODIFICADORES} \mid \dots$
- $\text{MODIFICADORES} \rightarrow \text{MODIFICADOR MODIFICADORES} \mid \text{CONJUNCION MODIFICADOR} \mid \text{MODIFICADOR}$
- $\text{MODIFICADOR} \rightarrow \text{MARCA} \mid \text{GAMA} \mid \text{ESPECIFICACION_TEC} \mid \text{ADJETIVO}$
- $\text{COMPLEMENTO} \rightarrow \text{USO} \mid \text{CONJUNCION USO}$

Además, la gramática verifica concordancia de género y número (por ejemplo, rechaza "una celular" o "el memoria") usando un sistema de unificación de rasgos implementado en unificacion.py.

2. Los autómatas implementados

2.1 ATN (atn.py)

Una vez que el parser construye el árbol sintáctico, el ATN lo recorre para extraer el significado de la frase: qué quiere el usuario, sobre qué dispositivo y con qué características. Su 5-tupla es:

- Q (estados): q_0_{inicio} , $q_1_{\text{en_intencion}}$, $q_2_{\text{en_requerim}}$, $q_{2a_en_articulo}$, $q_{2b_en_sujeto}$, $q_{2c_en_modif}$, $q_3_{\text{en_compl}}$, q_4_{aceptado} .
- Σ (entrada): los nodos del árbol DCG (INTENCION, REQUERIMIENTO, ARTICULO, SUJETO, MODIFICADORES, COMPLEMENTO).
- δ (transiciones): el autómata avanza nodo por nodo extrayendo información. Si un nodo opcional no aparece (por ejemplo, no hay COMPLEMENTO), usa un arco de POP para seguir sin bloquearse.
- q_0 (estado inicial): q_0_{inicio} .
- F (estado final): q_4_{aceptado} — se llega aquí cuando el árbol fue recorrido completamente.

Al terminar, el ATN deja registrados: la intención del usuario (buscar, recomendar...), el sujeto (celular, procesador...), los modificadores clasificados (marca, gama, especificación técnica) y el complemento de uso (jugar, programar...).

2.2 Manejo de Ambigüedad (ambigüedad.py)

A veces una frase admite más de una estructura válida (ambigüedad). Para elegir la mejor, usamos un autómata finito que evalúa y puntúa cada árbol candidato:

- **Q (estados):** q_start, q_evaluar_compl, q_evaluar_modifs, q_evaluar_intent, q_evaluar_sujeto, q_puntuar, q_comparar, q_aceptar.
- **Σ (entrada):** los árboles DCG candidatos.
- **δ (transiciones):** por cada candidato, el AF asigna puntos según si tiene COMPLEMENTO (+3), cuántos modificadores tiene (+1 por cada uno), si tiene INTENCION explícita (+2) y cuántos rasgos morfológicos tiene el sujeto (+1 por rasgo).
- **q₀ (estado inicial):** q_start.
- **F (estado final):** q_aceptar — se llega al seleccionar el candidato con mayor puntaje.

3. Cómo se integran las herramientas

El sistema funciona como un pipeline de 5 pasos encadenados:

- **Tokenización (utils.py):** la frase del usuario se normaliza y divide en tokens.
- **Parsing DCG (parser_dcg.py + gramatica.py):** se aplica la gramática para construir uno o varios árboles sintácticos válidos. Si ninguno encaja, se reporta el error de estructura o concordancia.
- **Desambiguación (ambigüedad.py):** si hay varios árboles válidos, el AF elige el que mejor capture la intención del usuario.
- **Análisis semántico (atn.py):** el ATN recorre el árbol elegido y extrae los registros semánticos (intención, sujeto, modificadores, uso).
- **Respuesta (generador_respuestas.py + base_datos.py):** con esos registros se consulta el catálogo de dispositivos y se presentan las recomendaciones al usuario.